Camada de Aplicação: Visão Geral

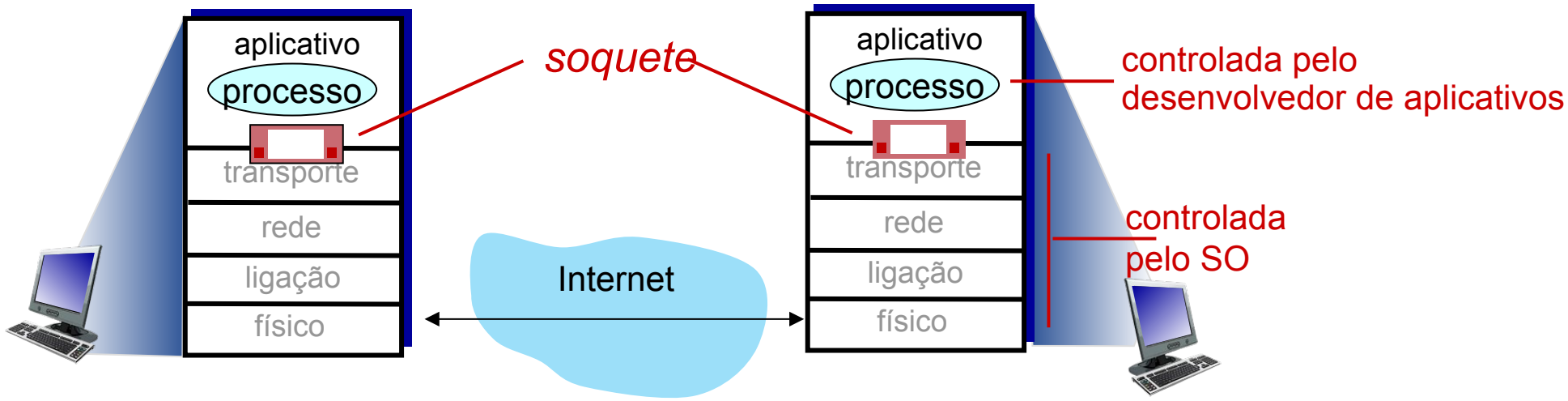
- Princípios das aplicações de rede
- Web e HTTP
- E-mail, SMTP, IMAP
- O Sistema de Nomes de Domínio (DNS)
- Aplicativos P2P
- Streaming de vídeo e redes de distribuição de conteúdo
- **Programação de soquetes com UDP e TCP**



Programação de sockets

objetivo: aprender a criar aplicativos cliente/servidor que se comunicam por meio de sockets

Socket: porta de comunicação entre o processo da aplicação e o protocolo de transporte de ponta a ponta



Programação de soquetes

Dois tipos de soquetes para dois serviços de transporte:

- *UDP:* datagrama não confiável
- *TCP:* confiável, orientado a fluxo de bytes

Exemplo de aplicação:

1. cliente lê uma entrada do teclado e envia-a ao servidor
2. servidor recebe os dados e converte os caracteres para maiúsculas
3. servidor envia os dados modificados ao cliente
4. cliente recebe os dados modificados e exibe-os

Programação de soquetes com UDP

UDP: não há "conexão" entre cliente e servidor:

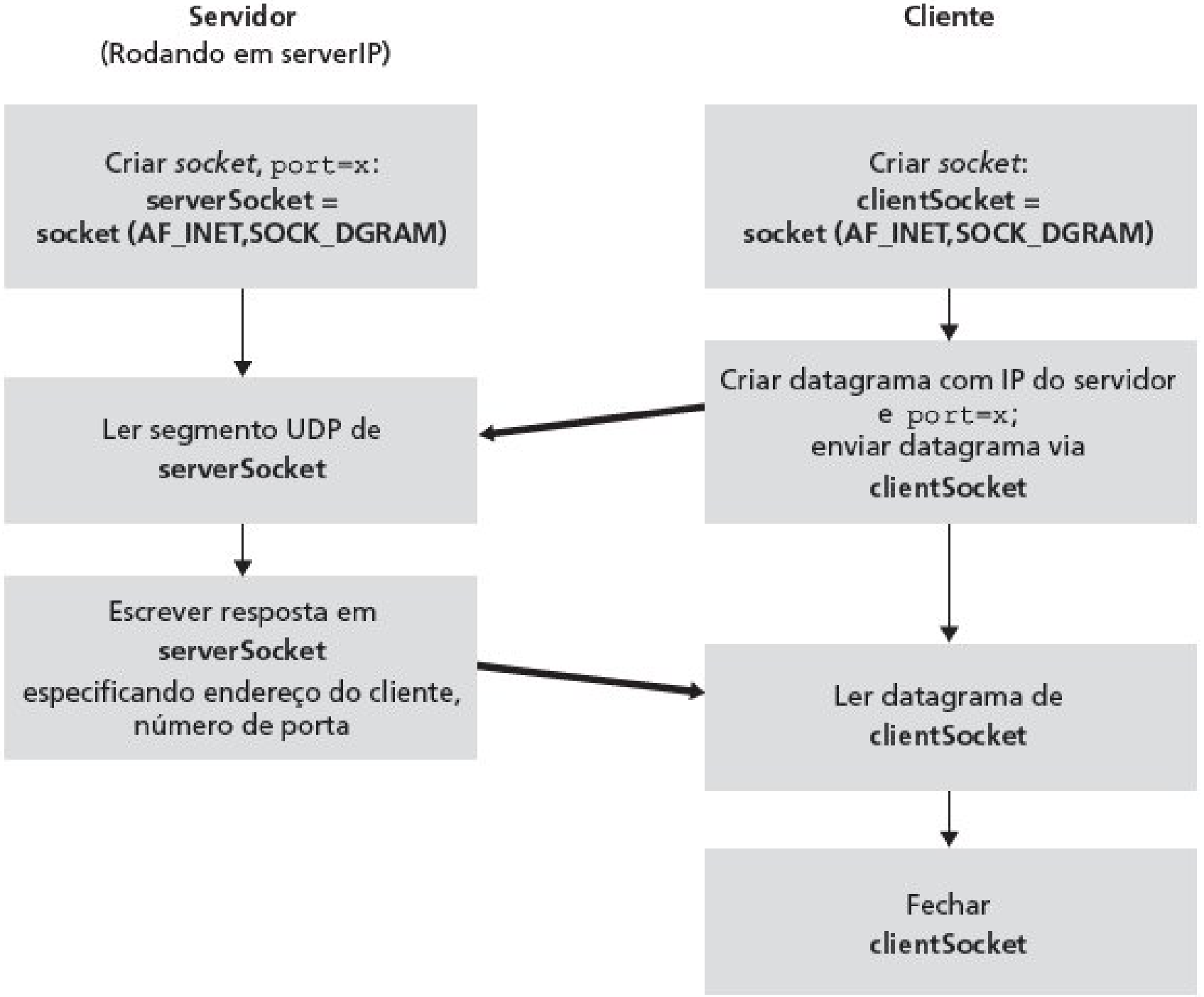
- não há handshaking antes do envio de dados
- remetente anexa explicitamente o endereço IP de destino e o número da porta a cada pacote
- receptor extrai o endereço IP e o número da porta do remetente do pacote recebido

UDP: os dados transmitidos podem ser perdidos ou recebidos fora de ordem

Do ponto de vista da aplicação:

- O UDP oferece transferência *não confiável* de grupos de bytes ("datagramas") entre processos de cliente e servidor

Interação entre soquetes cliente/servidor: UDP



Aplicativo de exemplo: cliente UDP

```python
from socket import *

nome_servidor = "localhost"
porta_servidor = 12000
socket_cliente = socket(AF_INET, SOCK_DGRAM)

mensagem = input("Digite algo (em caixa baixa): ")
socket_cliente.sendto(mensagem.encode(), (nome_servidor,porta_servidor))

resposta, endereco_servidor = socket_cliente.recvfrom(2048)
print("Resposta do servidor: ", resposta.decode())

socket_cliente.close()
```


which server is fastest??
facebook
Google
localhost


Network Computer não serve pra nada, o DEV:
mano 12:30
eu vou ficar rico programando 12:30
sério 12:30
olha meu projeto 12:30
http://localhost:3000/index.html
12:30


Não há lugar como
127.0.0.1
HostGator


BUT WHEN SHE TRACED THE KILLER'S IP ADDRESS... IT WAS IN THE 192.168/16 BLOCK!
GASP!

Aplicativo de exemplo: cliente UDP

```python
from socket import *

nome_servidor = "localhost"
porta_servidor = 12000
socket_cliente = socket(AF_INET, SOCK_DGRAM)

mensagem = input("Digite algo (em caixa baixa): ")
socket_cliente.sendto(mensagem.encode(), (nome_servidor,porta_servidor))

resposta, endereco_servidor = socket_cliente.recvfrom(2048)
print("Resposta do servidor: ", resposta.decode())

socket_cliente.close()
```

importa a biblioteca de sockets do Python

criar um socket UDP

entrada do usuário

anexa o nome do servidor e a porta à mensagem e envia para o soquete

recebe dados do socket

exibir a resposta

fecha o socket

Aplicativo de exemplo: cliente UDP

Python UDPClient

incluir a biblioteca de sockets do Python

```
from socket import *
serverName = 'hostname'
portaDoServidor = 12000
```

criar um socket UDP

```
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
```

obter a entrada do teclado do usuário

```
message = input('Digite uma frase em letras minúsculas:')
```

anexar o nome do servidor e a porta à mensagem; enviar para o soquete

```
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
```

ler os dados da resposta (bytes) do soquete

```
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
```

imprimir a string recebida e fechar o soquete

```
print(modifiedMessage.decode())
clientSocket.close()
```

Observação: esta atualização do código (2023) para Python 3

Aplicativo de exemplo: servidor UDP

```
1 from socket import *
2
3 porta_servidor = 12000
4 socket_servidor = socket(AF_INET, SOCK_DGRAM)
5 socket_servidor.bind( ("",porta_servidor) )
6 print("Servidor pronto para receber")
7
8 while True:
9     mensagem, endereco_cliente = socket_servidor.recvfrom(2048)
10    resposta = mensagem.decode().upper()
11    socket_servidor.sendto(resposta.encode(), endereco_cliente)
```

Aplicativo de exemplo: servidor UDP

```
1 from socket import *
2
3 porta_servidor = 12000
4 socket_servidor = socket(AF_INET, SOCK_DGRAM)
5 socket_servidor.bind( ("",porta_servidor) )
6 print("Servidor pronto para receber")
7
8 while True:
9     mensagem, endereco_cliente = socket_servidor.recvfrom(2048)
10    resposta = mensagem.decode().upper()
11    socket_servidor.sendto(resposta.encode(), endereco_cliente)
```

cria socket UDP

associa o socket à porta 12000 UDP

aguarda receber dados no socket

envia resposta ao cliente

Aplicativo de exemplo: servidor UDP

Servidor UDP em Python

```
from socket import *
portaDoServidor = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('O servidor está pronto para receber')
enquanto True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

criar soquete UDP

vincular o soquete à porta local número 12000

loop infinito

Ler do soquete UDP para a mensagem, obtendo o endereço do cliente (IP e porta do cliente)

enviar a string em maiúsculas de volta para esse cliente

Observação: esta atualização do código (2023) para Python 3

Exercícios

1. Teste os programas cliente e servidor demonstrados.
2. Modifique o programa servidor para que ele repasse cada mensagem recebida de um usuário A para um B e vice-versa.
3. Modifique o cliente dividindo-o em dois programas: cliente receptor e emissor. O receptor apenas receberá mensagens vindas do servidor enquanto que o emissor apenas enviará mensagens para o servidor. Ambos os programas devem ter um laço infinito.

   Dica: faça o cliente emissor enviar uma mensagem especial para que o servidor tome conhecimento de sua existência. Por exemplo: !REGISTRO
4. Modifique o cliente emissor para que seja solicitado um nome de usuário uma vez e envie-o sempre juntamente com a mensagem ao servidor.
5. Experimente conectar o seu cliente emissor ao servidor de um colega. Funcionou? Caso negativo, por que?
6. Faça uma versão P2P do programa de chat via UDP.